

Comparação de Algoritmos de Escalonamento de Processos com Prioridade Dinâmica por Envelhecimento e Estimativa de Rajada

Ângelo, Jetro, Luiz, Dorian

Curso de Engenharia de Software — Universidade Federal do Cariri (UFCA)

Disciplina de Sistemas Operacionais — Projeto da Unidade 3

Resumo — Este trabalho apresenta um simulador de escalonamento de processos por eventos discretos, escrito em C, para comparar três algoritmos clássicos — FCFS, Round Robin e Prioridade não preemptiva — com um algoritmo próprio, a Prioridade Dinâmica com Estimativa de Rajada e Envelhecimento. O algoritmo próprio combina prioridade estática, envelhecimento (aging) do tempo de espera e uma estimativa de rajada via média móvel exponencial (calculada apenas sobre rajadas já concluídas do próprio processo, sem uso de informação futura). Avaliamos os quatro algoritmos em 4 cenários obrigatórios (equilibrado, I/O-bound, CPU-bound e prioridades desbalanceadas), com 1000 processos por execução e 1000 seeds por cenário (10x o mínimo exigido), custo de troca de contexto maior que zero, e três análises complementares (custo de troca de contexto zero, sensibilidade de quantum do Round Robin e sensibilidade do peso de envelhecimento do algoritmo próprio). O resultado central é que o algoritmo próprio diferencia processos por prioridade de forma visível (razão de turnaround baixa/alta prioridade de 1,87x) sem reproduzir a quase-inanição da Prioridade estática pura (6,47x), obtendo justiça (índice de Jain do slowdown) muito acima da Prioridade, embora sem superar o turnaround médio agregado do FCFS.

Palavras-chave — escalonamento de processos; simulação a eventos discretos; envelhecimento (aging); Round Robin; índice de Jain; sistemas operacionais.

I. INTRODUÇÃO E MOTIVAÇÃO

Escalonar processos em um sistema operacional é, no fundo, um problema de arbitrar entre objetivos concorrentes: minimizar o tempo médio de resposta, tratar processos de forma justa e respeitar prioridades definidas externamente. Algoritmos clássicos otimizam bem para um desses objetivos às custas dos demais — escalonamento por Prioridade estática, por exemplo, favorece processos importantes mas pode deixar processos de baixa prioridade esperando indefinidamente (inanição); FCFS é inerentemente justo mas ignora qualquer noção de prioridade ou duração de rajada.

Este trabalho investiga se é possível obter parte da eficiência da Prioridade estática sem herdar seu problema de inanição, combinando-a com envelhecimento (aging) e uma estimativa da duração da próxima rajada de CPU. Construímos um simulador de escalonamento por eventos discretos em C, implementamos os três algoritmos clássicos exigidos (FCFS, Round Robin, Prioridade não preemptiva) e um algoritmo próprio, e comparamos os quatro sob quatro cargas de trabalho controladas por seed, com rigor estatístico (média e intervalo de confiança de 95% sobre 1000 execuções independentes por combinação cenário×algoritmo).

A contribuição deste artigo não é apenas o código do simulador, mas o processo de calibração do algoritmo próprio e a discussão honesta de seus resultados: mostramos que, com a fórmula aditiva simples que propomos, não existe um único peso de envelhecimento que supere o FCFS simultaneamente em turnaround médio e em justiça — e explicamos por quê (Seção VI). O código completo, os dados brutos e consolidados, e os scripts de análise estão disponíveis no repositório do projeto.

II. MODELO DO SISTEMA

Cada processo é modelado como uma sequência de rajadas $CPU \rightarrow E/S \rightarrow CPU \rightarrow E/S \rightarrow \dots \rightarrow CPU$, com

identificador, tempo de chegada, prioridade estática (convenção: menor valor numérico = maior prioridade) e uma lista de pares (duração de CPU, duração de E/S). Apenas a última rajada de um processo não é seguida de E/S. O tempo mínimo ideal de um processo — usado no cálculo do slowdown — é a soma de todas as suas durações de CPU e E/S, isto é, o tempo que ele levaria para terminar se nunca esperasse em nenhuma fila.

Entrada e saída (E/S). Um ou mais dispositivos (padrão: 1), cada um com fila FIFO própria e sem paralelismo interno. Um processo que solicita E/S é atribuído ao dispositivo de menor carga (ocupado + fila), com desempate round-robin entre dispositivos empatados. Ao concluir a E/S, o processo retorna imediatamente à fila de prontos.

Troca de contexto. Ocorre sempre que a CPU passa a executar um processo diferente do anterior, incluindo a transição ociosa→executando; tem custo configurável (>0 nos experimentos principais) durante o qual a CPU fica indisponível. Se o mesmo processo é redespachado sem que a CPU tenha ficado ociosa ou executado outro processo nesse intervalo, não há troca.

Chegadas. Processo modelado por intervalos entre chegadas exponencialmente distribuídos (média por cenário), em vez de todos os processos chegarem em $t=0$ — produz um regime de fila mais realista e evita favorecer artificialmente algoritmos que reordenam livremente uma fila já completa desde o início.

Detalhes completos de cada decisão de modelagem, incluindo a recalibração dos parâmetros do cenário I/O-bound (a primeira tentativa gerava utilização do dispositivo de E/S acima de 100%, fila sem limite e turnaround não-representativo), estão documentados em docs/mode_lagem.md no repositório.

III. ALGORITMOS DE ESCALONAMENTO

A. Algoritmos clássicos

FCFS: fila FIFO simples pela ordem de entrada na fila de prontos (chegada nova ou retorno de E/S). **Round Robin:** fila circular FIFO com quantum configurável (padrão 4); o processo cujo quantum expira antes de terminar a rajada retorna ao final da fila. **Prioridade não preemptiva:** escolhe sempre o processo pronto de menor valor de prioridade; desempate por menor tempo de chegada e, residualmente, menor identificador de processo. Nenhum dos três usa preempção fora da expiração de quantum do Round Robin.

B. Algoritmo próprio: Prioridade Dinâmica com Estimativa de Rajada e Envelhecimento

Não preemptivo. A cada CPU livre, escolhe o processo pronto de menor prioridade efetiva:

$$prioridade_efetiva(p, t) = prioridade(p) - k_1 \cdot espera(p, t) + k_2 \cdot rajada_estimada(p)$$

onde $espera(p, t)$ é o tempo desde que p entrou na fila de prontos pela última vez, e $rajada_estimada(p)$ é uma média móvel exponencial (EMA, fator $\alpha=0,5$) das durações de rajadas de CPU já concluídas do próprio processo — nunca a duração real, ainda desconhecida, da rajada em curso. Antes da primeira rajada concluída usa-se um valor de referência configurável (padrão 10,0).

Os pesos foram calibrados empiricamente testando $k_1 \in \{0,5; 0,05; 0,005\}$ nos 4 cenários (1000 seeds cada): $k_1=0,5$ é forte demais e faz o termo de envelhecimento dominar quase imediatamente frente às esperas médias observadas (dezenas a centenas de unidades de tempo), degenerando o algoritmo para um comportamento estatisticamente indistinguível do FCFS; $k_1=0,005$ é fraco demais e aproxima o comportamento da Prioridade estática pura. Adotamos $k_1=0,05$ e $k_2=0,3$ (este último com sensibilidade bem mais fraca — variação de 274,6 a 277,6 no turnaround médio do cenário equilibrado ao longo de $k_2 \in [0,1; 3,0]$) como configuração principal do artigo. Detalhes completos da calibração, incluindo a tabela de resultados por valor de k_1 , estão em docs/algoritmo_proprio.md.

A ideia de combinar envelhecimento com prioridade para evitar inanição, e de estimar a próxima rajada por média móvel exponencial sobre o histórico observado, são técnicas conhecidas na literatura de sistemas operacionais [1], [3]; nossa contribuição é a combinação específica dos dois mecanismos em uma fórmula aditiva com pesos calibrados empiricamente, e o diagnóstico de que essa fórmula não admite um ponto que domine o FCFS simultaneamente em turnaround e justiça (Seção VI).

IV. METODOLOGIA EXPERIMENTAL

Quatro cenários obrigatórios, cada um com carga de trabalho gerada deterministicamente a partir de uma seed (PRNG xorshift128+ próprio, independente de `rand()`/`srand()` da libc, para garantir reprodutibilidade entre máquinas): *equilibrado* (mistura de processos curtos/longos, pouca/muita E/S), *I/O-bound* (rajadas de CPU curtas, muita E/S), *CPU-bound* (rajadas longas, pouca E/S) e *prioridades desbalanceadas* (85% dos processos com prioridade alta, 15% com prioridade baixa).

Cada execução simula 1000 processos; cada combinação cenário×algoritmo foi repetida com 1000 seeds independentes (seeds 1–1000, as mesmas para todos os

algoritmos dentro de um cenário) — 10 vezes o mínimo de 100 seeds exigido, viável porque as ~45.000 execuções completas do experimento (principal + 3 análises complementares) levam poucos minutos. O custo de troca de contexto nos experimentos principais é 1 (>0); uma análise complementar repete tudo com custo 0 para isolar seu efeito.

Métricas obrigatórias: turnaround médio, número total de trocas de contexto, e índice de Jain aplicado ao slowdown de cada processo (slowdown = turnaround / tempo mínimo ideal). Para cada métrica, reportamos média entre seeds e intervalo de confiança de 95% (média amostral $\pm 1,96 \cdot s/\sqrt{n}$, desvio padrão amostral). Adicionalmente, quebramos o turnaround médio por classe de prioridade (alta: ≤ 3 ; baixa: ≥ 8) para medir o quanto cada algoritmo de fato diferencia processos por prioridade.

V. RESULTADOS

A Fig. 1 (próxima página) resume as três métricas obrigatórias nos 4 cenários. O Round Robin (quantum=4) tem turnaround médio e número de trocas de contexto uma ordem de grandeza acima dos demais algoritmos em todos os cenários exceto I/O-bound — investigamos essa diferença na Seção VI. Entre os outros três, FCFS e o algoritmo próprio ficam próximos em turnaround agregado, com a Prioridade obtendo o menor turnaround médio à custa de justiça muito inferior (painel c).

Tabela I. Turnaround médio (unidades de tempo)

Cenário	FCFS	RR	Prio.	Próprio
Equilibrado	305,8	2.277,3	267,0	305,8
I/O-bound	143,5	146,7	143,2	143,0
CPU-bound	296,0	3.384,6	271,1	306,1
Prio. desbal.	305,1	2.255,1	265,6	305,3

Tabela II. Índice de Jain do slowdown (%)

Cenário	FCFS	RR	Prio.	Próprio
Equilibrado	43,1	71,6	7,4	37,9
I/O-bound	70,3	71,0	70,1	70,1
CPU-bound	56,6	77,2	15,7	55,0
Prio. desbal.	42,9	71,6	7,6	38,0

A Tabela III isola o cenário de prioridades desbalanceadas por classe de prioridade — o resultado mais importante deste trabalho (ver também Fig. 2, próxima página):

Tabela III. Turnaround por classe de prioridade — cenário prioridades desbalanceadas

Algoritmo	Prio. alta	Prio. baixa	Razão
FCFS	305,2	304,7	1,00×
RR	2.255,3	2.253,9	1,00×
Prioridade	146,2	946,2	6,47×
Próprio	270,0	505,4	1,87×

Comparacao entre algoritmos por cenario (media $\pm$ IC95%, 1000 seeds/cenario)

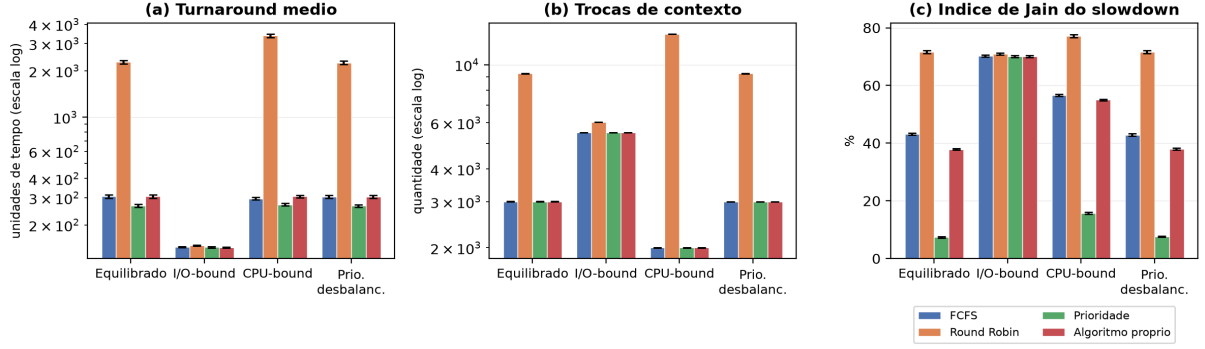


Fig. 1. Turnaround médio, trocas de contexto (escala log) e índice de Jain do slowdown, por cenário e algoritmo. Média $\pm$ IC95%, 1000 seeds/cenário, 1000 processos/execução, custo de troca de contexto=1.

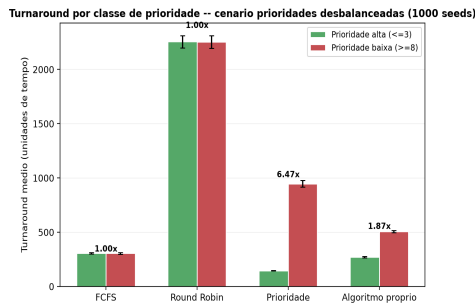


Fig. 2. Turnaround médio por classe de prioridade (alta: prioridade ≤ 3 ; baixa: ≥ 8) no cenário de prioridades desbalanceadas, com a razão baixa/alta anotada acima de cada par de barras. FCFS não diferencia (1,00x); Prioridade estática discrimina fortemente (6,47x); o algoritmo próprio fica entre os dois extremos (1,87x). Média $\pm$ IC95%, 1000 seeds.

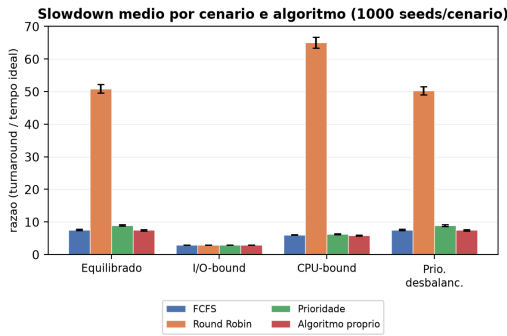


Fig. 3. Slowdown médio por cenário e algoritmo (métrica auxiliar). Média $\pm$ IC95%, 1000 seeds/cenário.

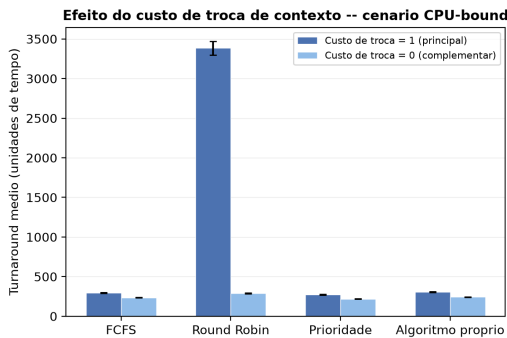


Fig. 4. Turnaround médio em CPU-bound com custo de troca de contexto 1 (principal) vs. 0 (complementar).

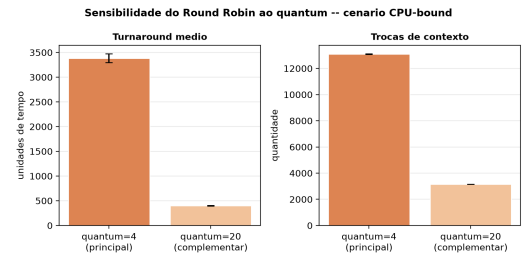


Fig. 5. Round Robin em CPU-bound: quantum=4 (principal) vs. quantum=20 (complementar).

As Figs. 3–4 (inline abaixo) mostram as duas análises complementares: efeito do custo de troca de contexto e sensibilidade do Round Robin ao quantum.

VI. DISCUSSÃO

A. Round Robin: turnaround e trocas de contexto muito piores

No cenário CPU-bound (rajadas de CPU longas, média 25 unidades de tempo), o Round Robin com quantum=4 tem turnaround médio de $3.384,64 \pm 86,39$, contra $295,95 \pm 5,26$ do FCFS — e 13.094 ± 16 trocas de contexto contra 2.000 ± 2 do FCFS (mesmo número de rajadas totais). Cada rajada de ~ 25 unidades é fatiada em cerca de 6 quanta de 4 unidades, e cada fatia exige uma nova troca; sob carga alta (utilização de CPU $\approx 83\%$ neste cenário) isso empurra o RR para perto da saturação: com quantum=4, o tempo total consumido por trocas de contexto chega a rivalizar com o próprio tempo de CPU disponível. Repetindo o experimento com quantum=20 (mais próximo da rajada média), o turnaround cai para $399,84 \pm 8,54$ e as trocas de contexto para 3.160 ± 4 — uma melhora de quase uma ordem de grandeza (Fig. 5), confirmando que o problema é a relação quantum/rajada, não uma limitação estrutural do Round Robin.

B. Efeito do custo de troca de contexto

Com custo de troca de contexto zero (análise complementar), o turnaround do RR em CPU-bound cai de $3.384,64 \pm 86,39$ para $289,33 \pm 4,33$ — uma redução muito mais acentuada, em termos proporcionais, do que a observada nos outros três algoritmos (ex.: FCFS vai de $295,95 \pm 5,26$ para $234,68 \pm 3,33$, uma queda bem mais modesta). Isso é esperado: o RR gera muito mais trocas de contexto que os demais (Tabela I/ Fig. 1b), então o custo por troca pesa proporcionalmente mais no seu turnaround total. O experimento confirma que grande parte da desvantagem do RR neste cenário vem do *custo acumulado das trocas*, não apenas da ordem de execução

em si (Fig. 4).

C. O algoritmo próprio diferencia por prioridade sem inanição extrema

O resultado mais nítido do projeto é a Tabela III / Fig. 2: no cenário de prioridades desbalanceadas, o FCFS trata processos de prioridade alta e baixa de forma praticamente idêntica (razão 1,00× — ignora completamente a prioridade, por construção), enquanto a Prioridade estática discrimina de forma severa (6,47×: processos de baixa prioridade terminam, em média, mais de 6 vezes mais devagar que os de alta prioridade). O algoritmo próprio fica entre os dois extremos (1,87×): usa a prioridade de forma visível, mas o termo de envelhecimento evita que processos de baixa prioridade sejam deixados para trás indefinidamente — seu turnaround médio na classe de baixa prioridade (505) é quase metade do observado na Prioridade pura (946). O mesmo padrão qualitativo aparece no cenário CPU-bound com custo de troca zero (prioridade uniforme, não desbalanceada): razões de 1,00×, 4,92× e 1,67× respectivamente — não é um artefato de um único cenário.

D. O algoritmo próprio não supera o FCFS na média agregada — e por quê

Apesar do resultado da seção anterior, o turnaround médio *agregado* (todos os processos, não só por classe) do algoritmo próprio fica estatisticamente indistinguível do FCFS na maioria dos cenários (Tabela I), e sua justiça agregada (Tabela II) fica sistematicamente abaixo da do FCFS (ex.: 37,9% contra 43,1% no cenário equilibrado). **Entre os algoritmos não preemptivos avaliados** (FCFS, Prioridade e o algoritmo próprio), o FCFS obtém o maior índice de Jain nos 4 cenários, por ordenar estritamente por tempo de espera sem privilegiar rajadas curtas ou prioridades altas — qualquer regra que se desvie dessa ordenação pura para perseguir eficiência (prioridade, rajada estimada) reintroduz alguma assimetria dentro dessa família. Essa leitura **não se generaliza** para o conjunto completo dos 4 algoritmos: o Round Robin — que é preemptivo — obtém um índice de Jain *maior* que o do FCFS nos 4 cenários (ex.: 71,6% contra 43,1% no cenário equilibrado; Tabela II/Fig. 1c), já que intercalar a execução entre todos os processos prontos reduz por construção a variância relativa de slowdown entre eles. A conclusão sobre maior simetria, portanto, fica restrita à comparação entre os algoritmos **não preemptivos** avaliados neste trabalho, e não se estende ao Round Robin nem, necessariamente, a algoritmos preemptivos em geral. A varredura de k_1 (Seção III-B) reforça a leitura dentro dessa família não preemptiva: à medida que k_1 diminui e o algoritmo se afasta do comportamento FCFS-símile, o turnaround agregado melhora muito pouco (300,32 contra 305,76 no cenário equilibrado, $k_1=0,005$ vs. 0,05) enquanto a justiça cai visivelmente mais — não encontramos, nessa fórmula aditiva linear, um ponto que domine o FCFS nas duas métricas ao mesmo tempo, dentro da família não preemptiva. O valor real do algoritmo, à luz dos nossos experimentos, está na Seção VI-C: ele entrega quase toda a robustez do FCFS contra inanição enquanto ainda usa o sinal de prioridade de forma clara — algo que nem FCFS nem Prioridade pura fazem ao mesmo tempo.

VII. CONCLUSÃO

Implementamos um simulador de escalonamento por eventos discretos com quatro algoritmos e o avaliamos sob quatro cargas de trabalho controladas por seed, com rigor estatístico substancialmente acima do mínimo exigido (1000 seeds/cenário). Confirmamos empiricamente um resultado clássico de sistemas operacionais — Round Robin com

quantum pequeno frente a rajadas longas gera overhead de troca de contexto suficiente para dominar o turnaround sob carga alta — e quantificamos precisamente o efeito (queda de quase uma ordem de grandeza no turnaround ao aumentar o quantum de 4 para 20). Para o algoritmo próprio, o achado central é que ele oferece um ponto de equilíbrio genuíno entre Prioridade estática e FCFS na dimensão de “quanto a prioridade importa” (razão de turnaround baixa/alta prioridade de 1,87×, entre o 1,00× do FCFS e o 6,47× da Prioridade pura), mesmo não superando o FCFS em turnaround agregado — um resultado que só ficou visível porque medimos turnaround por classe de prioridade, não apenas a média geral. Como trabalho futuro, destacamos duas direções concretas: (i) uma função de envelhecimento não-linear ou normalizada pelo tempo mínimo ideal do próprio processo, que dispense recalibração de k_1 a cada regime de carga; e (ii) uma variante preemptiva do algoritmo próprio, para tratar o caso de uma rajada de baixa prioridade muito longa bloquear um processo de prioridade alta recém-chegado.

REFERÊNCIAS

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] R. Jain, D. Chiu, and W. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems,” DEC Research Report TR-301, Digital Equipment Corporation, 1984.
- [3] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An Experimental Time-Sharing System,” in *Proc. AFIPS Spring Joint Computer Conference*, 1962, pp. 335–344.